

Loka — iOS Architecture & Component Plan

1. Technical Direction

Primary Platforms

Native iOS application.

Native Android application.

UI Frameworks

iOS

SwiftUI.

Android

Jetpack Compose.

Architecture Style

Feature-oriented modular architecture.

Recommended pattern:

- MVVM
- Service Layer
- Repository Layer
- Centralized App State where necessary

Avoid premature overengineering.

2. Core Mobile App/Client Modules

Authentication Module

Responsibilities:

- OTP login
 - Session handling
 - Secure token storage
 - Logout
-

Citizen Verification Module

Responsibilities:

- Aadhaar Offline XML upload
 - Share-code entry
 - Verification processing
 - Verification status handling
 - District selection
-

Feed Module

Responsibilities:

- Nearby Issues
 - New Issues
 - Community Priority
 - Resolved Issues
 - Feed pagination
-

Issue Module

Responsibilities:

- Issue detail
 - Support/Oppose participation
 - Evidence display
 - Related issues
 - Issue updates
-

Create Issue Module

Responsibilities:

- Issue creation form
 - Validation
 - Duplicate detection
 - Evidence upload
 - Draft handling
-

Search Module

Responsibilities:

- Keyword search
 - Geographic filtering
 - Category filtering
 - Status filtering
-

Notifications Module

Responsibilities:

- Notification list
 - Read/unread state
 - Notification routing
-

Profile Module

Responsibilities:

- Citizen profile
 - Participation history
 - Verification status
 - Participation regions
 - Account settings
-

3. Shared Core Components

App Router

Central navigation management.

Responsibilities:

- Deep-link routing
 - Authentication gating
 - Navigation state
-

API Client

Responsibilities:

- HTTP requests
 - Authentication headers
 - Token refresh
 - Error handling
 - Request retry policy
-

Secure Storage

Use Keychain for:

- Access tokens
- Refresh tokens
- Sensitive session metadata

Android equivalent should use secure encrypted storage.

App Session Manager

Responsibilities:

- Login state
- Citizen state

- Verification state
- Region state

Upload Manager

Responsibilities:

- Evidence uploads
- XML uploads
- Upload progress
- Retry handling

4. Suggested Folder Structure

Loka/ ├── App/ ├── Core/ ├── Services/ ├── Repositories/ ├── Models/
├── Features/ | ├── Authentication/ | ├── Verification/ | ├── Feed/ |
├── Issue/ | ├── CreateIssue/ | ├── Search/ | ├── Notifications/ |
└── Profile/ ├── DesignSystem/ ├── Utilities/ └── Resources/

5. Design System Structure

Typography

Shared text styles.

Examples:

- HeadingLarge
 - HeadingMedium
 - Body
 - Caption
 - StatusLabel
-

Colors

Color system should remain calm and neutral.

Avoid:

- Aggressive reds
- Viral-style gradients
- Entertainment styling

Suggested direction:

- Neutral backgrounds
 - Civic blue/green accents
 - Soft status indicators
-

Shared Components

Buttons

- Primary Button
 - Secondary Button
 - Destructive Warning Button
-

Issue Card

Reusable across:

- Feed
 - Search
 - Related Issues
 - Profile
-

Status Badge

Examples:

- Under Review
- Active

- Resolved
 - Archived
 - Rejected
-

Confirmation Modal

Used for:

- Support
 - Oppose
 - Permanent actions
 - Destructive actions
-

Empty State View

Reusable empty-state screens.

Loading States

Skeleton loading for:

- Feed
 - Issue details
 - Search results
-

6. Data Model Direction

Immutable Participation

Support/Oppose actions should be immutable in client state.

The UI should reflect permanence clearly.

Optimistic Updates

Recommended for:

- Support
- Oppose
- Comments

But:

- must reconcile with backend confirmation
 - must handle moderation restrictions gracefully
-

7. Offline & Caching Strategy

Initial Direction

Basic offline support only.

Examples:

- Cached feed
- Cached issue details
- Draft persistence

Avoid full offline complexity in V1.

8. Push Notification Direction

Notification Types:

- Issue approved
- Issue rejected
- Clarification requested
- Participation received
- Appeal updates
- Resolution updates

Push notifications should remain utility-focused.

Avoid addictive re-engagement tactics.

9. Security Direction

Sensitive Data Handling

Never expose:

- Aadhaar details
 - Verification internals
 - Internal moderation metadata
-

Session Security

- Short-lived access tokens
 - Refresh-token rotation
 - Secure logout handling
-

API Security

- Authenticated endpoints
 - Geographic participation enforcement
 - Rate limiting
 - Abuse protection
-

10. Backend Architecture Direction

Recommended Starting Style

Modular monolith.

Reason:

- Faster development

- Easier operations
- Simpler debugging
- Lower infrastructure complexity

Do not start with microservices.

11. Suggested Backend Modules

- Authentication
 - Citizen Verification
 - Citizen Management
 - Issue Management
 - Participation Engine
 - Comment System
 - Moderation Engine
 - Notification Service
 - Audit Logging
 - Reporting & Safety
-

12. Storage Direction

Relational Database

Recommended for:

- participation integrity
 - moderation auditability
 - geographic relationships
 - transactional consistency
-

Media Storage

Separate object storage for:

- issue evidence

- uploaded media
 - verification uploads
-

13. Logging & Monitoring

Track:

- API failures
- Verification failures
- Upload failures
- Moderation bottlenecks
- Participation anomalies

Auditability is critical.

14. Engineering Principles

Principle 1

Correctness over speed.

Principle 2

Operational simplicity over architectural hype.

Principle 3

Security before convenience.

Principle 4

Transparent systems over hidden automation.

Principle 5

Build for trust first.

15. Cross-Platform Product Consistency

Shared Across Platforms

The following must remain consistent across iOS and Android:

- Participation rules
- Moderation behavior
- Verification flow
- Geographic restrictions
- Issue lifecycle
- API contracts
- Validation rules
- Core navigation philosophy

Minor UI adaptation for platform conventions is acceptable.

Behavior divergence is not acceptable.

Shared System Foundations

Shared between platforms:

- Backend APIs
 - Business logic
 - Database models
 - Moderation rules
 - Product specifications
 - Design system guidelines
-

Platform Responsibilities

iOS

- SwiftUI implementation
- Native platform optimization
- Apple ecosystem integration

Android

- Jetpack Compose implementation
 - Android ecosystem optimization
 - Broad device compatibility
-

Development Coordination Requirement

Because Loka supports dual-native development from V1:

- API contracts must be finalized before feature implementation
 - Shared product behavior documentation must remain updated
 - Feature parity between platforms should remain closely aligned
-

16. Recommended Immediate Engineering Steps

Step 1

Setup repositories and project structure.

Step 2

Build authentication flow.

Step 3

Build citizen verification flow.

Step 4

Implement issue creation and feed.

Step 5

Implement support/opposition participation.

Step 6

Build moderation dashboard.

Step 7

Begin controlled pilot testing.

17. MVP Engineering Goal

The engineering goal of V1 is not scale.

The goal is:

- stable participation
- trustworthy verification
- accountable moderation
- operational learning
- product validation

before large-scale expansion.